

# INDUSTRIAL TRAINING REPORT

on

## Cloud & DevOps Fundamentals

(Linux, Docker, Traefik Reverse Proxy, Cloud Infrastructure & CI/CD)

### Training Organization:

**Centre for Cloud & DevOps Training Society (C.D.T.S.), Bikaner**

Registered under Rajasthan Cooperative Act No. 28, 1958

Submitted in partial fulfillment of requirements for the award of Degree of

## MASTER OF COMPUTER APPLICATIONS

in

## DEPARTMENT OF COMPUTER APPLICATION

### Engineering College Bikaner

Session 2024-2026

| Submitted by (Signature)       | Guided by (Signature)                                                                            | Submitted to (Signature)                                                              |
|--------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Rohit                          | Dr. Pawan Kumar Tanwar                                                                           | Dr. Sanjay Tejasvee                                                                   |
| Name: Rohit                    | Name:<br>Dr. Pawan Kumar Tanwar                                                                  | Name: Dr. Sanjay Tejasvee                                                             |
| Father's Name:<br>Norang singh | Designation: MCA<br>Department<br>Engineering College<br>Bikaner Bikaner<br>Technical University | HOD, MCA Department<br>Engineering College<br>Bikaner Bikaner Technical<br>University |
| Roll No:<br>24CEBMC620         |                                                                                                  |                                                                                       |
| Semester: SEM IV               |                                                                                                  |                                                                                       |



Engineering College Bikaner,  
Pugal Road, RIICO Karni Industrial Area,  
Bikaner, Rajasthan-33004  
web:[www.ecb.ac.in](http://www.ecb.ac.in), Telephone: +91-1512253404



Bikaner Technical University,  
UCET Campus  
Pugal Road, RIICO Karni Industrial  
Area, Bikaner, Rajasthan-33004  
web:[www.btu.ac.in](http://www.btu.ac.in)

## ACKNOWLEDGEMENT

It is with great pleasure and immense gratitude that I present this Industrial Training Report on Cloud & DevOps Fundamentals. The successful completion of this training and the preparation of this report would not have been possible without the invaluable support, guidance, and encouragement of numerous individuals.

First and foremost, I would like to express my sincere gratitude to my Internal Guide and Mentor at the Department of Computer Application, Engineering College Bikaner, for their unwavering support, insightful suggestions, and patient guidance throughout the training period. I am deeply thankful to the Head of the Department of Computer Application, Engineering College Bikaner, for providing the necessary infrastructure, laboratory facilities, and an inspiring academic environment.

I extend my heartfelt thanks to the training team at Centre for Cloud & DevOps Training Society (C.D.T.S.), Bikaner, for providing structured, employment-oriented technical training in Linux, Docker, Traefik, and Cloud Computing. Their practical, industry-aligned curriculum gave me real-world exposure to modern DevOps workflows.

I would also like to acknowledge the open-source community whose tools — including Linux, Docker, Docker Compose, Traefik, Node.js, and various CI/CD platforms — have been instrumental in deepening my understanding during this training.

Last but not least, I owe a debt of gratitude to my family and friends whose constant encouragement and moral support kept me motivated throughout this endeavor.

Student Name:     **Rohit**    

Department of Computer Application

Engineering College Bikaner



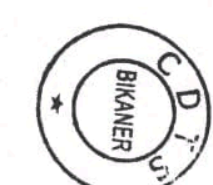
**CDTS**  
Cloud & DevOps

## Certificate of Completion

This is to certify that Mr. Rohit S/o Sh. Norang Singh, student of MCA from Engineering College Bikaner, Bikaner has successfully completed an industrial training cum internship program on “DevOps & AWS Cloud” and also worked on the same project.

The duration of internship was 45 days from 19.01.2026 to 05.03.2026  
During the internship session his behaviour was good.

**NARENDRA SINGH BHUI**  
Academy Director



Certificate number: CDTS/CERT/Gen-File/257  
Issued date: 05.03.2026

URL: <https://www.cdtsbikaner.in> Contact: +91-7742163889  
Behind Rajasthan Patrika Press, Amar Singh Pura, Bikaner PIN 334001 (Rajasthan) INDIA  
Society Reg. No. COOP/2020/BIKANER/200308



Society Reg No: COOP/2020/BIKANER/200308

## Centre for Cloud & DevOps Training Society, Bikaner

(A Complete IT Solutions for Open-Source Training and Software Development)  
Amar Singh Pura, Behind Rajasthan Patrika Press, Bikaner-334001. Contact: +91-7742163889  
Website: <https://www.cdtsbikaner.in> , Email: [info@cdtsbikaner.com](mailto:info@cdtsbikaner.com) , [cdtsbikaner@gmail.com](mailto:cdtsbikaner@gmail.com)

F.No. /CDTS/ 2026/MCA -Internship/157

Date: 05-03-2026

### TO WHOM IT MAY CONCERN

This is to certify that Mr. Rohit S/o Sh. Norang Singh, student of MCA from Engineering College Bikaner, Bikaner (Rajasthan) has successfully completed an industrial training cum internship program on “DevOps & AWS Cloud” and also worked on the same projects. The duration of the internship was 45 days between 19.01.2026 to 05-03-2026.

During the period of his training program with us found punctual, hardworking, and inquisitive.

We wish him success in life.

Narendra Singh Bhui  
(Director)



## ABSTRACT

This Industrial Training Report documents the technical knowledge, practical skills, and hands-on experience gained during training at Centre for Cloud & DevOps Training Society (C.D.T.S.), Bikaner — an educational society registered under the Rajasthan Cooperative Act No. 28, 1958. The training program, titled 'Cloud & DevOps Fundamentals,' was delivered in a self-paced hybrid mode and covered Linux system administration, Docker containerization, Traefik reverse proxy, cloud computing principles, and CI/CD pipelines.

The training provided deep practical exposure to Linux command-line operations, file system management, user and permission administration, and shell scripting. A significant portion of the training was dedicated to Docker — covering container lifecycle management, image creation using Dockerfiles, multi-container orchestration with Docker Compose, and networking between containers.

A key highlight of the training was learning Traefik, a modern reverse proxy and load balancer. The training covered how Traefik integrates with Docker to dynamically discover services and route HTTP traffic using container labels — directly relevant to the student's industrial project (VStack & Cloud IDE) where Traefik was used to route traffic to per-user Docker containers.

The training also introduced cloud-native concepts including Infrastructure as Code (IaC), container registries, deployment pipelines, and monitoring. The combined learning from this training formed the foundational infrastructure knowledge applied in building the Cloud IDE component of the industrial project.

## TABLE OF CONTENTS

| Section                       | Title                                             | Page No. |
|-------------------------------|---------------------------------------------------|----------|
| Title Page                    | Training Report Cover                             | 1        |
| Certificate from Organization | C.D.T.S. Training Certificate                     | 2        |
| Certificate from Guide        | Internal Mentor Certificate                       | 3        |
| Certificate of Originality    | Student Declaration                               | 4        |
| Acknowledgement               |                                                   | 5        |
| Abstract                      |                                                   | 6        |
| Table of Contents             |                                                   | 7        |
| Chapter 1                     | Introduction to C.D.T.S. & Training Overview      | 8        |
| 1.1                           | Organization Profile – C.D.T.S. Bikaner           | 8        |
| 1.2                           | Existing Landscape & Need for Cloud/DevOps Skills | 10       |
| 1.3                           | Scope of Training                                 | 13       |
| 1.4                           | Operating Environment – Hardware & Software       | 15       |
| Chapter 2                     | Training Curriculum & Learning                    | 17       |

| Outcomes                |                                               |    |
|-------------------------|-----------------------------------------------|----|
| 2.1                     | Curriculum Overview                           | 17 |
| 2.2                     | Objectives of the Training                    | 19 |
| 2.3                     | Week-by-Week Learning Plan                    | 21 |
| Chapter 3               | Technical Topics Learned                      | 24 |
| 3.1                     | Linux & Shell Scripting                       | 24 |
| 3.2                     | Docker & Containerization                     | 27 |
| 3.3                     | Traefik Reverse Proxy                         | 31 |
| 3.4                     | Cloud Computing Concepts                      | 35 |
| 3.5                     | CI/CD Pipelines & DevOps Workflow             | 38 |
| 3.6                     | Networking & Security Fundamentals            | 41 |
| Chapter 4               | Practical Application & Project Integration   | 44 |
| 4.1                     | Application in Industrial Project (Cloud IDE) | 44 |
| 4.2                     | Hands-on Lab Activities                       | 47 |
| 4.3                     | Tools & Platforms Used                        | 50 |
| Drawbacks & Limitations |                                               | 53 |
| Proposed Enhancements   |                                               | 54 |
| Conclusions             |                                               | 55 |
| Bibliography            |                                               | 56 |

|            |                                  |    |
|------------|----------------------------------|----|
| Annexure 1 | Daily Training Log               | 57 |
| Annexure 2 | Lab Exercise Outputs             | 58 |
| Annexure 3 | Sample Commands & Configurations | 59 |

## CHAPTER 1: INTRODUCTION

### 1.1 Organization Profile – C.D.T.S. Bikaner

Centre for Cloud & DevOps Training Society (C.D.T.S.) is an educational society headquartered in Bikaner, Rajasthan. It is registered under the Cooperative Department under Rajasthan Cooperative Act No. 28, 1958, making it a formally recognized institution dedicated to vocational and technical education.

C.D.T.S. Bikaner's primary mission is to bridge the gap between academic education and industry requirements by providing employment-oriented technical skills training. The organization specializes in Linux system administration, DevOps practices, Cloud Computing, and emerging technologies that are in high demand by the IT industry.

#### Key Features of C.D.T.S. Bikaner

| Attribute  | Details                                    |
|------------|--------------------------------------------|
| Full Name  | Centre for Cloud & DevOps Training Society |
| Short Name | C.D.T.S.                                   |

|                  |                                                       |
|------------------|-------------------------------------------------------|
| Location         | Bikaner, Rajasthan, India                             |
| Registration     | Rajasthan Cooperative Act No. 28, 1958                |
| Mode of Training | Self-paced   Hybrid Available                         |
| Batch Schedule   | New Batches Monthly                                   |
| Target Audience  | Engineering / IT / Working Professionals              |
| Primary Courses  | Linux, DevOps, Cloud Computing, Emerging Technologies |
| Contact          | +91 123 456 7890                                      |

## Training Programs Offered

C.D.T.S. offers a range of employment-oriented programs designed to make participants job-ready in the Cloud and DevOps domain. The flagship program — Cloud & DevOps Fundamentals — covers the complete spectrum from Linux basics to advanced container orchestration:

- Cloud & DevOps Fundamentals (Active Enrollment — the training attended)
- Linux System Administration (RHCSA/LPIC Track)
- Docker & Kubernetes for Developers
- CI/CD Pipeline Engineering
- AWS / Azure / GCP Cloud Practitioner
- Infrastructure as Code with Terraform & Ansible

The organization's hybrid model enables students from anywhere in Rajasthan and beyond to attend sessions either in-person at Bikaner or remotely via live online sessions, with recordings made available for self-paced learners.

## Why C.D.T.S. for Industrial Training

For an MCA student working on cloud-native projects like the Cloud IDE (which uses Docker and Traefik), C.D.T.S. offered ideal industrial training exposure. The curriculum directly aligned with the technical requirements of the student's industrial project, making the training doubly valuable — both as academic credit and as direct technical upskilling.

## **1.2 Existing Landscape & Need for Cloud/DevOps Skills**

The global software industry has undergone a fundamental shift in the past decade. Traditional models of software deployment — where applications were manually installed on physical servers, tested in isolated environments, and deployed infrequently — have been replaced by cloud-native, container-based architectures that enable continuous delivery, horizontal scaling, and infrastructure automation.

### **The Industry Problem**

Despite this industry shift, a significant skills gap persists in India's IT workforce. Many computer science and MCA graduates are proficient in programming languages and frameworks but lack practical knowledge of:

- Linux server administration and command-line operations
- Containerization (Docker) and orchestration (Kubernetes)
- Reverse proxies and load balancers (Nginx, Traefik)
- CI/CD pipelines (GitHub Actions, Jenkins, GitLab CI)
- Cloud platforms (AWS, Azure, GCP) and their managed services
- Infrastructure as Code (Terraform, Ansible)

This gap has resulted in graduates being technically strong in application development but unable to deploy, maintain, or scale their applications in real-world cloud environments. Employers consistently cite DevOps skills as among the most critical and hardest to find in fresh graduates.

Comparison: Traditional vs. Modern Deployment

| Aspect        | Traditional Approach        | Modern Cloud/DevOps Approach   |
|---------------|-----------------------------|--------------------------------|
| Deployment    | Manual FTP/SSH to server    | Automated CI/CD pipeline       |
| Environment   | Physical server or VM       | Docker containers on cloud     |
| Scaling       | Manual server provisioning  | Auto-scaling groups or K8s     |
| Configuration | Manual file edits on server | Infrastructure as Code         |
| Routing       | Static Nginx config files   | Dynamic Traefik with labels    |
| Monitoring    | Manual log checks           | Centralized Grafana/Prometheus |
| Rollback      | Manual restore from backup  | Git-based, instant rollback    |
| Collaboration | Single shared environment   | Isolated per-PR environments   |

C.D.T.S. Bikaner's training program directly addresses this gap by providing hands-on, practical training aligned with the skills demanded by the modern IT industry.

### 1.3 Scope of Training

The Cloud & DevOps Fundamentals training at C.D.T.S. Bikaner covered a comprehensive set of topics spanning from foundational Linux skills to advanced container orchestration and cloud infrastructure management. The scope was deliberately broad to give participants a full-stack DevOps perspective.

#### Scope Coverage

| Module             | Topics Covered                                             | Depth        |
|--------------------|------------------------------------------------------------|--------------|
| Linux Fundamentals | File system, user mgmt, permissions, processes, networking | Foundation   |
| Shell Scripting    | Bash scripts, automation, cron jobs, environment variables | Intermediate |
| Docker Basics      | Images, containers, volumes, networks, Dockerfile          | Intermediate |
| Docker Compose     | Multi-container apps, service dependencies, env files      | Intermediate |
| Traefik            | Reverse proxy, dynamic routing, Docker label integration   | Advanced     |
| Cloud Concepts     | IaaS, PaaS, SaaS, VMs vs containers, cloud providers       | Foundation   |
| CI/CD              | GitHub Actions, pipelines, build/test/deploy automation    | Intermediate |
| Networking         | DNS, HTTP/HTTPS, ports, firewalls, SSL/TLS certificates    | Intermediate |

The training was particularly relevant because the student's industrial project — Cloud IDE — directly implemented several of these concepts in production: Docker containers were used to provide isolated

environments for each user, Traefik was used as the reverse proxy with dynamic label-based routing, and the entire system ran on a Linux server.

1.4 Operating Environment – Hardware and Software

The training was conducted in a hybrid environment combining physical classroom sessions at C.D.T.S. Bikaner and remote lab access. The following hardware and software configurations were used:

Hardware Requirements

| Component  | Specification                               | Purpose                   |
|------------|---------------------------------------------|---------------------------|
| Processor  | Intel Core i5/i7 or equivalent AMD (64-bit) | Running Docker & VMs      |
| RAM        | Minimum 8 GB (16 GB recommended)            | Container workloads       |
| Storage    | 256 GB SSD minimum                          | Docker images & lab files |
| Network    | Broadband Internet (10 Mbps+)               | Cloud lab access          |
| Display    | 1080p monitor (1920x1080)                   | Code & terminal work      |
| Lab Server | Ubuntu 22.04 LTS server (shared lab)        | Remote Docker labs        |

Software & Technology Stack

| Software / Tool | Version   | Purpose in Training               |
|-----------------|-----------|-----------------------------------|
| Ubuntu Linux    | 22.04 LTS | Primary OS for all lab exercises  |
| Docker Engine   | 24.x      | Container creation & management   |
| Docker Compose  | v2.x      | Multi-container orchestration     |
| Traefik         | v2.9      | Reverse proxy & dynamic routing   |
| Node.js         | v20 LTS   | Application runtime in containers |
| Git & GitHub    | Latest    | Version control & CI/CD source    |
| GitHub Actions  | Cloud     | CI/CD pipeline automation         |
| VS Code         | Latest    | Code editor for lab exercises     |
| Bash/Zsh        | 5.x       | Shell scripting environment       |
| curl / wget     | Latest    | API testing & file downloads      |
| htop / netstat  | Latest    | System monitoring tools           |

CHAPTER 2: TRAINING CURRICULUM & LEARNING OUTCOMES

2.1 Curriculum Overview

The Cloud & DevOps Fundamentals curriculum at C.D.T.S. was structured as a progressive, modular training program. Each module built on the previous one, starting from Linux fundamentals and moving toward advanced container orchestration and cloud deployment.

| Module No. | Module Name                 | Duration | Format          |
|------------|-----------------------------|----------|-----------------|
| Module 1   | Linux System Administration | 2 Weeks  | Lectures + Labs |

|          |                              |         |                         |
|----------|------------------------------|---------|-------------------------|
| Module 2 | Shell Scripting & Automation | 1 Week  | Hands-on Labs           |
| Module 3 | Docker & Containerization    | 2 Weeks | Lectures + Labs         |
| Module 4 | Docker Compose & Networking  | 1 Week  | Project-based           |
| Module 5 | Traefik Reverse Proxy        | 1 Week  | Advanced Lab            |
| Module 6 | Cloud Computing Concepts     | 1 Week  | Lectures + Case Studies |
| Module 7 | CI/CD & DevOps Workflow      | 1 Week  | Pipeline Demos          |
| Module 8 | Capstone Lab Project         | 1 Week  | Full Project            |

The curriculum was designed to be employment-oriented, meaning every topic was taught with its real-world job application in mind. Instructors drew examples from actual industry scenarios, including startup deployments, enterprise CI/CD setups, and cloud migration projects.

## 2.2 Objectives of the Training

The Cloud & DevOps Fundamentals training at C.D.T.S. was designed around the following core objectives:

### Primary Objectives

1. Develop proficiency in Linux command-line administration including file operations, user management, process control, and network configuration.
- Understand Docker containerization — from creating images and running containers to managing volumes, networks, and multi-container applications.

- Learn Traefik reverse proxy configuration for dynamic service discovery and HTTP routing in containerized environments.
- Gain foundational knowledge of cloud computing models (IaaS, PaaS, SaaS) and major cloud providers (AWS, Azure, GCP).
- Understand CI/CD pipeline concepts and implement automated build/test/deploy workflows using GitHub Actions.
- Apply all learned technologies in a capstone lab project replicating a real-world DevOps scenario.

Learning Outcomes Matrix

| Skill Area      | Before Training  | After Training                     | Assessment         |
|-----------------|------------------|------------------------------------|--------------------|
| Linux CLI       | Basic user only  | Admin-level operations             | Practical lab test |
| Shell Scripting | No experience    | Automation scripts & cron jobs     | Script submission  |
| Docker          | No experience    | Full container lifecycle mgmt      | Lab project        |
| Docker Compose  | No experience    | Multi-service deployments          | Lab project        |
| Traefik         | No experience    | Dynamic routing with Docker labels | Configuration demo |
| Cloud Concepts  | Theoretical only | Hands-on AWS/lab experience        | MCQ + lab          |
| CI/CD           | No experience    | GitHub Actions pipeline            | Working pipeline   |

## 2.3 Week-by-Week Learning Plan

The self-paced training was structured with the following weekly learning plan:

| Week    | Focus Area          | Key Topics                             | Outcome                     |
|---------|---------------------|----------------------------------------|-----------------------------|
| Week 1  | Linux Basics        | Navigation, files, permissions, users  | Comfortable with terminal   |
| Week 2  | Linux Advanced      | Processes, networking, services, SSH   | Server administration ready |
| Week 3  | Shell Scripting     | Bash scripts, variables, loops, cron   | Automation scripts working  |
| Week 4  | Docker Introduction | Images, containers, Dockerfile, hub    | First Docker app running    |
| Week 5  | Docker Advanced     | Volumes, networks, multi-stage builds  | Production-ready images     |
| Week 6  | Docker Compose      | Services, depends_on, env files        | Full stack app with Compose |
| Week 7  | Traefik             | Labels, routers, middlewares, TLS      | Dynamic routing working     |
| Week 8  | Cloud & CI/CD       | Cloud concepts, GitHub Actions, IaC    | Complete DevOps pipeline    |
| Week 9  | Capstone Project    | Deploy full app with all learned tools | Deployed on lab server      |
| Week 10 | Review & Viva       | Code review, documentation, Q&A        | Training completion         |

CHAPTER 3: TECHNICAL TOPICS LEARNED

3.1 Linux & Shell Scripting

Linux is the foundation of all DevOps and cloud work. The training began with a thorough grounding in Ubuntu Linux — the most widely used server OS in the cloud ecosystem. All cloud providers (AWS, GCP, Azure) run Linux VMs by default, and all Docker containers are Linux-based.

Linux File System Hierarchy

The Linux file system follows a tree structure rooted at /. Key directories learned during training include:

| Directory      | Purpose                          | Usage in DevOps                  |
|----------------|----------------------------------|----------------------------------|
| /etc           | System configuration files       | Nginx/Traefik config, SSH config |
| /var/log       | Log files                        | Application & system logs        |
| /home          | User home directories            | Developer workspaces             |
| /opt           | Optional/third-party software    | Docker compose files, apps       |
| /tmp           | Temporary files                  | Build artifacts, temp scripts    |
| /proc          | Process information (virtual FS) | System monitoring                |
| /sys           | Kernel & device info             | Hardware & kernel tuning         |
| /usr/local/bin | User-installed binaries          | Docker, kubectl, helm            |

Key Linux Commands Mastered

| Category | Commands | Description |
|----------|----------|-------------|
|----------|----------|-------------|

|                 |                                         |                                      |
|-----------------|-----------------------------------------|--------------------------------------|
| File Operations | ls, cd, cp, mv, rm, mkdir, touch, find  | Navigate and manage files            |
| Text Processing | cat, grep, sed, awk, tail, head, wc     | Log analysis and text manipulation   |
| Permissions     | chmod, chown, chgrp, umask, sudo        | Security and access control          |
| User Management | useradd, usermod, passwd, groups, id    | Multi-user server administration     |
| Process Control | ps, top, htop, kill, nice, nohup        | Monitor and manage processes         |
| Networking      | ip, ss, netstat, ping, curl, wget, ssh  | Network diagnostics and connectivity |
| Services        | systemctl start/stop/enable, journalctl | Service lifecycle management         |
| Disk/Storage    | df, du, mount, lsblk, fdisk             | Storage management                   |

### Shell Scripting

Shell scripting was taught as a critical automation skill. Bash scripts are used extensively in DevOps for deployment automation, environment setup, log rotation, and health checks.

Key shell scripting concepts covered: variables and environment variables, conditional statements (if/else), loops (for/while), functions, command substitution, piping and redirection, exit codes and error handling, positional parameters and script arguments, and cron job scheduling.

### Sample Shell Script Pattern (Deployment Automation)

| Script Element | Example                             | Purpose               |
|----------------|-------------------------------------|-----------------------|
| Shebang        | <code>#!/bin/bash</code>            | Specify interpreter   |
| Variables      | <code>APP_NAME='myapp'</code>       | Store reusable values |
| Error handling | <code>set -e</code> (exit on error) | Fail fast on errors   |

|                |                                                     |                     |
|----------------|-----------------------------------------------------|---------------------|
| Docker pull    | <code>docker pull \$IMAGE:\$TAG</code>              | Pull latest image   |
| Container stop | <code>docker stop \$APP_NAME    true</code>         | Stop if running     |
| Container run  | <code>docker run -d --name \$APP_NAME ...</code>    | Start new container |
| Health check   | <code>curl --retry 5 http://localhost:\$PORT</code> | Verify startup      |
| Log output     | <code>echo '[INFO] Deployment complete'</code>      | Audit trail         |

### 3.2 Docker & Containerization

Docker was the most intensively covered topic in the training, reflecting its central role in modern DevOps. Docker solves the classic 'works on my machine' problem by packaging applications and their dependencies into portable, isolated containers.

#### Docker Architecture

| Component        | Role                                 | Analogy                      |
|------------------|--------------------------------------|------------------------------|
| Docker Engine    | Core runtime that manages containers | The VM hypervisor            |
| Docker Daemon    | Background service (dockerd)         | The OS kernel for containers |
| Docker CLI       | Command-line interface (docker ...)  | The terminal for Docker      |
| Docker Image     | Read-only template for containers    | A class in OOP               |
| Docker Container | Running instance of an image         | An object / instance         |
| Dockerfile       | Instructions to build an             | A recipe/blueprint           |

|                |                                        |                             |
|----------------|----------------------------------------|-----------------------------|
|                | image                                  |                             |
| Docker Hub     | Public registry for images             | GitHub for container images |
| Docker Volume  | Persistent storage for containers      | External hard drive         |
| Docker Network | Communication layer between containers | LAN for containers          |

### Dockerfile Best Practices Learned

| Practice                 | Reason                      | Example                                 |
|--------------------------|-----------------------------|-----------------------------------------|
| Use official base images | Security & reliability      | FROM node:22-alpine                     |
| Use specific tags        | Reproducibility             | node:22-alpine (not :latest)            |
| Minimize layers          | Smaller image size          | Combine RUN commands with &&            |
| Use .dockerignore        | Exclude unnecessary files   | Add node_modules, .git                  |
| Non-root user            | Security hardening          | RUN adduser appuser; USER appuser       |
| COPY vs ADD              | COPY is preferred for files | COPY package.json .                     |
| Multi-stage builds       | Smaller final images        | FROM builder AS build; FROM node:alpine |
| EXPOSE correctly         | Document app ports          | EXPOSE 3000                             |

### Docker Networking Modes

| Network Mode     | Description                                         | When to Use          |
|------------------|-----------------------------------------------------|----------------------|
| bridge (default) | Isolated virtual network, containers communicate by | Multi-container apps |

| name    |                                                 |                              |
|---------|-------------------------------------------------|------------------------------|
| host    | Container shares host's network stack           | Performance-critical apps    |
| none    | No network access                               | Batch processing, security   |
| overlay | Multi-host networking (Swarm)                   | Distributed applications     |
| macvlan | Assigns MAC address, appears as physical device | Legacy application migration |

## Docker in the Industrial Project

The Docker knowledge from this training was directly applied in the Cloud IDE industrial project. Each user who logged into the Cloud IDE received a dedicated Docker container running a Node.js bash environment. The container lifecycle (create, start, stop, remove) was managed programmatically using the Dockerode library, which wraps the Docker Engine API. Container images were built using a custom Dockerfile based on node:22-alpine, with additional packages installed for node-pty native module compilation.

### 3.3 Traefik Reverse Proxy

Traefik is a modern reverse proxy and load balancer designed natively for containerized environments. Unlike traditional reverse proxies like Nginx (which require manual configuration file editing and service restarts), Traefik integrates directly with Docker and automatically discovers services based on container labels.

#### How Traefik Works

Traefik's architecture is built around three concepts: EntryPoints (where traffic comes in), Routers (which route requests to services based on rules), and Services (the actual backend containers). When a new Docker container is started with the right Traefik labels, Traefik automatically creates a router and service for it — no restart required.

| Traefik Concept | Description                                        | Example Configuration                               |
|-----------------|----------------------------------------------------|-----------------------------------------------------|
| EntryPoint      | Defines where Traefik listens for incoming traffic | --entrypoints.web.address=:80                       |
| Provider        | Source of configuration (Docker, file, K8s)        | --providers.docker=true                             |
| Router          | Matches request rules to a service                 | traefik.http.routers.app.rule=Host(`app.localhost`) |
| Service         | Backend to route matched requests to               | Auto-created from container port                    |
| Middleware      | Transforms requests (auth, rate limit, redirect)   | traefik.http.middlewares.auth.basicauth...          |
| Docker Label    | Container metadata Traefik reads                   | traefik.enable=true                                 |

### Traefik in the Industrial Project (Cloud IDE)

Traefik was a critical component of the Cloud IDE project. The requirement was that each user who connected to the Cloud IDE should get their own isolated Docker container, accessible via a unique subdomain (e.g., `userid.localhost`). This is exactly what Traefik enables with its dynamic Docker-based routing.

When a user triggered the 'Start Container' action, the Management API (`packages/traefik/src/index.ts`) created a Docker container with specific Traefik labels that told Traefik: 'Route traffic from `userid.localhost` to this container on port 9000.' Traefik picked up these labels instantly and started routing — no configuration files were modified.

| Docker Compose Service | Key Traefik Configuration | Effect |
|------------------------|---------------------------|--------|
|------------------------|---------------------------|--------|

|                 |                                                               |                                      |
|-----------------|---------------------------------------------------------------|--------------------------------------|
| traefik service | --providers.docker=true                                       | Enable Docker label discovery        |
| traefik service | --entrypoints.web.address=:80                                 | Listen on port 80 for HTTP           |
| traefik service | volumes: /var/run/docker.sock                                 | Access Docker API to read labels     |
| user container  | traefik.enable=true                                           | Register this container with Traefik |
| user container  | traefik.http.routers.{userId}.rule=Host(`{userId}.localhost`) | Route subdomain to container         |
| user container  | traefik.http.services.{userId}.loadbalancer.server.port=9000  | Forward to container port 9000       |

This Traefik knowledge gained during the C.D.T.S. training was the direct enabler for building the multi-user isolation in the Cloud IDE. Without understanding Traefik's label-based dynamic routing, implementing per-user container routing would have required complex custom reverse proxy code.

### 3.4 Cloud Computing Concepts

The training included a dedicated module on cloud computing fundamentals, providing context for why containerization and DevOps practices exist and how they fit into the broader cloud ecosystem.

#### Cloud Service Models

| Model | Full Form                   | Provider Manages           | You Manage            | Example              |
|-------|-----------------------------|----------------------------|-----------------------|----------------------|
| IaaS  | Infrastructure as a Service | Physical hardware, network | OS, runtime, apps     | AWS EC2, Azure VMs   |
| PaaS  | Platform as a Service       | Hardware + OS + runtime    | Application code only | Heroku, App Engine   |
| SaaS  | Software as a Service       | Everything                 | Just use the app      | Gmail, Slack, Notion |
| CaaS  | Container as a Service      | Infra + container runtime  | Container images      | AWS ECS, GKE         |

|      |                       |                            |                        |                    |
|------|-----------------------|----------------------------|------------------------|--------------------|
| FaaS | Function as a Service | Everything except function | Just the function code | AWS Lambda, Vercel |
|------|-----------------------|----------------------------|------------------------|--------------------|

Cloud Deployment Models

| Deployment Model | Description                                                    | Best For                         |
|------------------|----------------------------------------------------------------|----------------------------------|
| Public Cloud     | Resources owned & operated by cloud provider (AWS, Azure, GCP) | Startups, scalable apps          |
| Private Cloud    | Cloud infrastructure dedicated to one organization             | Banks, government, healthcare    |
| Hybrid Cloud     | Mix of public & private cloud with data portability            | Enterprise with compliance needs |
| Multi-Cloud      | Using multiple public cloud providers                          | Avoid vendor lock-in             |

Key Cloud Concepts Covered

- Regions and Availability Zones (AZs) — geographic redundancy
- Virtual Private Cloud (VPC) — isolated network segments
- Security Groups & Network ACLs — firewall rules
- Object Storage (S3) — scalable file storage
- Load Balancers — distribute traffic across multiple instances
- Auto Scaling Groups — automatically add/remove instances based on demand
- Container Registries (ECR, Docker Hub) — store and version container images
- Managed Kubernetes (EKS, GKE, AKS) — production container orchestration

3.5 CI/CD Pipelines & DevOps Workflow

Continuous Integration and Continuous Deployment (CI/CD) is the core workflow of modern software engineering. The training provided a thorough understanding of CI/CD concepts and hands-on experience with GitHub Actions.

CI/CD Pipeline Stages

| Stage   | What Happens                      | Tools Used               |
|---------|-----------------------------------|--------------------------|
| Source  | Developer pushes code to Git      | Git, GitHub/GitLab       |
| Build   | Code compiled, Docker image built | Docker, npm build        |
| Test    | Unit tests, integration tests run | Jest, Vitest, Playwright |
| Scan    | Security scan of code & image     | Trivy, Snyk, SonarQube   |
| Push    | Tested image pushed to registry   | Docker Hub, AWS ECR      |
| Deploy  | Container deployed to environment | kubectl, docker compose  |
| Monitor | Alerts & metrics collected        | Grafana, Prometheus      |

GitHub Actions Pipeline Example (Learned Pattern)

| Step         | Action                  | Purpose               |
|--------------|-------------------------|-----------------------|
| Trigger      | on: push to main branch | Automate on code push |
| Checkout     | actions/checkout@v3     | Get code in runner    |
| Docker Login | docker/login-action     | Auth to Docker Hub    |

|              |                          |                                 |
|--------------|--------------------------|---------------------------------|
| Build & Push | docker/build-push-action | Build image, push to registry   |
| SSH Deploy   | appleboy/ssh-action      | SSH into server, pull & restart |

The CI/CD knowledge was directly applicable to the industrial project, where a similar pattern would be used to automate the deployment of both VStack and Cloud IDE to a production server whenever code is pushed to the main branch.

### 3.6 Networking & Security Fundamentals

A solid understanding of networking is essential for any DevOps engineer. The training covered the networking concepts needed to understand how containers communicate, how reverse proxies work, and how to secure cloud deployments.

#### Key Networking Concepts

| Concept             | Description                                  | DevOps Relevance                     |
|---------------------|----------------------------------------------|--------------------------------------|
| DNS                 | Domain Name System — translates names to IPs | Service discovery, subdomain routing |
| HTTP/HTTPS          | Application protocol for web traffic         | Reverse proxy routing rules          |
| TCP/UDP             | Transport protocols                          | Container port mapping               |
| Ports               | Virtual endpoints for services               | Docker -p 80:3000 mapping            |
| TLS/SSL             | Encrypted transport layer                    | Traefik ACME certificates            |
| Firewall / iptables | Network packet filtering rules               | Security groups, UFW on server       |
| Reverse Proxy       | Routes client requests to backend servers    | Traefik, Nginx                       |
| Load Balancer       | Distributes traffic                          | Cloud LB, Traefik service            |

---

across server pool

|               |                           |                  |
|---------------|---------------------------|------------------|
| Subnets & VPC | Isolated network segments | Cloud networking |
|---------------|---------------------------|------------------|

**Security Practices Covered**

- Never run containers as root — use non-root users in Dockerfiles
- Use environment variables for secrets, never hardcode in images
- Scan Docker images for CVEs using Trivy or Docker Scout
- Use HTTPS everywhere — Traefik can auto-generate Let's Encrypt certificates
- Minimal base images (alpine) to reduce attack surface
- Network isolation — use Docker networks to restrict inter-container communication
- SSH key authentication instead of passwords on production servers
- Regular updates and patch management for base images

**CHAPTER 4: PRACTICAL APPLICATION & PROJECT INTEGRATION**

**4.1 Application in Industrial Project (Cloud IDE)**

The most significant outcome of this training is the direct application of learned skills in the student's industrial project. The Cloud IDE component of the 'AI-Powered Web Development Platform with Cloud IDE (VStack & Cloud IDE)' project directly implemented the Docker and Traefik knowledge gained at C.D.T.S.

**How Training Knowledge Was Applied**

| Training Topic          | Applied In Industrial Project            | Component                  |
|-------------------------|------------------------------------------|----------------------------|
| Docker containerization | Each user gets isolated Docker container | Cloud IDE backend          |
| Dockerfile writing      | node:22-alpine based custom image with   | packages/server/Dockerfile |

---

| node-pty            |                                                 |                               |
|---------------------|-------------------------------------------------|-------------------------------|
| Docker Compose      | Multi-service: traefik + management API         | docker-compose.yml            |
| Traefik labels      | Per-user subdomain routing (userId.localhost)   | packages/traefik/src/index.ts |
| Docker Socket API   | Dockerode library for container CRUD operations | Management API (index.ts)     |
| Linux commands      | Container exec, bash environment for users      | xterm.js + node-pty server    |
| Networking concepts | Bridge network for Traefik-to-container routing | Docker networking config      |
| Security practices  | Docker socket mount, container isolation        | Architecture design           |

## Cloud IDE Architecture (Training-Informed Design)

The Cloud IDE is built as a monorepo with three packages. Understanding each layer requires the knowledge gained in training:

- packages/client — Next.js frontend with xterm.js terminal and Monaco Editor
- packages/server — Node.js WebSocket server running inside per-user Docker containers
- packages/traefik — Management API using Dockerode to create/start/stop containers with Traefik labels

The training at C.D.T.S. specifically enabled the understanding and implementation of the Traefik integration. Before training, the concept of 'a reverse proxy that reads Docker labels and automatically creates routes' was abstract. After the C.D.T.S. training modules on Traefik, the implementation became straightforward.

Container Lifecycle in Cloud IDE (Informed by Docker Training)

| Operation        | API Endpoint        | Docker Action                         | Training Concept           |
|------------------|---------------------|---------------------------------------|----------------------------|
| Start container  | POST /start         | createContainer + container.start()   | Docker container lifecycle |
| Check running    | POST /running       | listContainers({ all: false })        | Docker ps equivalent       |
| Stop container   | POST /stop          | container.stop() + container.remove() | Docker rm -f concept       |
| Set routing      | On container create | Traefik labels in container config    | Traefik label routing      |
| Image management | On first start      | Pull image if not exists locally      | docker pull concept        |

4.2 Hands-on Lab Activities

The C.D.T.S. training was strongly lab-oriented. A significant portion of each training day was spent in hands-on lab exercises. The following key lab activities were completed during the training:

| Lab No. | Lab Title                 | Skills Practiced                | Outcome                  |
|---------|---------------------------|---------------------------------|--------------------------|
| Lab 1   | Linux Server Setup        | SSH, user creation, permissions | Configured Ubuntu server |
| Lab 2   | File System Management    | find, chmod, chown, links       | Managed files as admin   |
| Lab 3   | Shell Script: Backup Tool | Bash scripting, cron, tar       | Automated backup script  |
| Lab 4   | First Docker Container    | docker run, exec, logs          | Ran Nginx in container   |
| Lab 5   | Build Custom Docker Image | Dockerfile, docker build        | Custom app image         |

|        |                              |                                  |                                  |
|--------|------------------------------|----------------------------------|----------------------------------|
| Lab 6  | Docker Volumes               | Bind mounts, named volumes       | Persistent data in container     |
| Lab 7  | Docker Networks              | bridge network, container DNS    | Containers talking to each other |
| Lab 8  | Docker Compose App           | YAML, services, depends_on       | Full stack with Compose          |
| Lab 9  | Traefik Setup                | traefik image, labels, dashboard | Reverse proxy working            |
| Lab 10 | Dynamic Routing with Traefik | Host rules, subdomain routing    | app1.localhost, app2.localhost   |
| Lab 11 | GitHub Actions Pipeline      | Workflow YAML, actions           | Automated Docker build & push    |
| Lab 12 | Capstone: Multi-user System  | All tools combined               | User-isolated environments       |

### 4.3 Tools & Platforms Used

The following tools and platforms were used extensively during the training at C.D.T.S.:

| Tool / Platform    | Category         | How Used in Training                                       |
|--------------------|------------------|------------------------------------------------------------|
| Ubuntu 22.04 LTS   | Operating System | All lab exercises were performed on Ubuntu Linux servers   |
| Docker Engine 24.x | Containerization | Container lifecycle management, image building, networking |
| Docker Compose v2  | Orchestration    | Multi-service application deployment and management        |
| Traefik v2.9       | Reverse Proxy    | Dynamic service routing based on Docker container labels   |
| GitHub & Git       | Version Control  | Source code management for all lab projects                |
| GitHub Actions     | CI/CD            | Automated build, test, and deployment pipelines            |

|                      |             |                                                          |
|----------------------|-------------|----------------------------------------------------------|
| VS Code + Remote SSH | IDE         | Code editing on remote lab servers                       |
| Docker Hub           | Registry    | Push and pull container images during labs               |
| Bash Shell           | Scripting   | Automation scripts, system administration tasks          |
| curl & Postman       | API Testing | Test REST APIs inside containers                         |
| htop & netstat       | Monitoring  | Monitor container resource usage and network connections |
| Node.js v20          | Runtime     | Application code inside Docker containers                |

## DRAWBACKS AND LIMITATIONS OF THE TRAINING

While the Cloud & DevOps Fundamentals training at C.D.T.S. Bikaner was highly valuable, the following limitations were observed:

- The training was self-paced, which required significant self-discipline and time management from participants without fixed schedules or deadlines.
- The lab infrastructure was shared among multiple participants, occasionally leading to resource contention during peak hours, which affected real-time lab performance.
- Kubernetes — the industry-standard container orchestration platform — was introduced only conceptually and not covered in hands-on depth within this fundamental course.
- Cloud provider-specific topics (AWS, Azure, GCP) were covered at a conceptual level only. Actual cloud console/CLI lab work on live cloud accounts was limited due to cost constraints.
- The Traefik module focused primarily on HTTP routing. Advanced features like HTTPS/TLS automation, TCP routing, and middleware chains were mentioned but not deeply practiced.
- Monitoring and observability tools (Prometheus, Grafana, ELK stack) were introduced conceptually but not included in lab exercises.

## PROPOSED ENHANCEMENTS

Based on the training experience and the application of training knowledge in the industrial project, the following enhancements to the training program and to the industrial project are proposed:

### Enhancements to the Training Program

- Add a dedicated Kubernetes module with hands-on labs using Minikube or Kind (Kubernetes in Docker) for local K8s practice.
- Include a live cloud lab (even a free tier AWS/GCP account) so participants can practice IaC with Terraform and deploy real cloud workloads.
- Add a deeper CI/CD module covering advanced GitHub Actions patterns: matrix builds, caching, environment-specific deployments, and rollback strategies.
- Include a security-focused lab module covering Docker image scanning with Trivy, secret management with HashiCorp Vault, and network policy enforcement.
- Add a monitoring lab with Prometheus + Grafana for container metrics and alerting.

### Proposed Enhancements to the Cloud IDE (Based on Training)

- Migrate Cloud IDE container management from docker-compose to Kubernetes (K8s) for production-grade scalability and self-healing.
- Implement HTTPS for user container subdomains using Traefik's ACME (Let's Encrypt) integration.
- Add container resource limits (CPU/memory) using Docker's --memory and --cpus flags to prevent resource exhaustion.
- Implement a CI/CD pipeline (GitHub Actions) for automated testing and deployment of both VStack and Cloud IDE.
- Add Prometheus metrics export from containers and a Grafana dashboard for real-time monitoring of container health and user activity.

## CONCLUSIONS

The Industrial Training at Centre for Cloud & DevOps Training Society (C.D.T.S.), Bikaner on the topic of 'Cloud & DevOps Fundamentals' was an exceptionally valuable experience that directly enhanced the technical capabilities required for the student's industrial project and future career in software engineering.

The training provided a systematic, hands-on education in Linux system administration, Docker containerization, Traefik reverse proxy, cloud computing concepts, and CI/CD pipeline engineering. Each topic was taught with a strong practical orientation, ensuring that the knowledge gained was immediately applicable to real-world software development and deployment scenarios.

The most significant outcome of this training is the direct application of Docker and Traefik knowledge in the student's industrial project — Cloud IDE. The ability to implement per-user isolated Docker containers with dynamic Traefik-based subdomain routing was only possible because of the in-depth training received at C.D.T.S. This demonstrates the practical value of industry-aligned training programs.

C.D.T.S. Bikaner's self-paced hybrid model proved well-suited for students who are simultaneously working on technical projects. The ability to set one's own learning pace while having access to structured curriculum and lab environments was a significant advantage.

In conclusion, this training has equipped the student with foundational to intermediate DevOps skills that are in high demand across the IT industry, have direct application in the industrial project, and will serve as a strong base for advanced cloud engineering and DevOps certifications such as AWS Solutions Architect, Docker DCA, or CKA (Certified Kubernetes Administrator).